

DSA0502 – Query Processing for Data Science

CO5 AT2 – INDUSTRY PROBLEM SOLVING

Course Faculty: Dr. B .SHYAMALA GOWRI **Reg. No.:** 192324164

Student Name: Sweety Roselin A **Date:** 17/08/2026

PROJECT 4: FINANCIAL MARKET ANALYSIS

QUESTION

How do financial prices evolve over time, what relationships exist between trading volume and daily returns, and what market patterns or dependencies can be identified using distributions, scatter plots, scatter matrices, lag plots, and autocorrelation plots?

AIM

To analyze historical financial market data using Pandas and visualization techniques, study price evolution and the relationship between trading volume and daily returns, identify market patterns and possible dependencies using statistical plots, and summarize useful findings for investment analysts.

DATASET

Dataset Name: Historical Stock Market Dataset

Source: Representative Open Financial Market Dataset

Number of Records: 20

Variables:

- Date
- Opening Price
- Closing Price
- Trading Volume
- Daily Return

Dataset Values

S.No	Date	Opening Price	Closing Price	Trading Volume	Daily Return
1	2025-01-01	100.00	101.50	12000	1.50
2	2025-01-02	101.50	102.20	13500	0.69
3	2025-01-03	102.20	101.80	11000	-0.39
4	2025-01-04	101.80	103.40	15000	1.57
5	2025-01-05	103.40	104.10	14200	0.68

6	2025-01-06	104.10	103.20	16000	-0.86
7	2025-01-07	103.20	105.00	17500	1.74
8	2025-01-08	105.00	106.30	18000	1.24
9	2025-01-09	106.30	105.70	15500	-0.56
10	2025-01-10	105.70	107.20	19000	1.42
11	2025-01-11	107.20	108.50	21000	1.21
12	2025-01-12	108.50	107.80	18500	-0.65
13	2025-01-13	107.80	109.60	22000	1.67
14	2025-01-14	109.60	110.20	23000	0.55
15	2025-01-15	110.20	108.90	25000	-1.18
16	2025-01-16	108.90	111.50	27000	2.39
17	2025-01-17	111.50	113.00	29000	1.35
18	2025-01-18	113.00	112.10	26000	-0.80
19	2025-01-19	112.10	114.50	31000	2.14
20	2025-01-20	114.50	118.00	40000	3.06

CODE

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from pandas.plotting import (
```

```
    scatter_matrix,
```

```
    lag_plot,
```

```
    autocorrelation_plot
```

```
)
```

```
# Read dataset
```

```
df = pd.read_csv("financial_market.csv")
```

```
# Convert date
```

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
print("FINANCIAL MARKET DATASET")
```

```
print(df)
```

```
# -----
```

```
# 1. Price Evolution
```

```
# -----
```

```
plt.figure(figsize=(12,5))
```

```
plt.plot(
```

```
    df["Date"],
```

```
    df["Opening_Price"],
```

```
    marker="o",
```

```
    label="Opening Price"
```

```
)
```

```
plt.plot(
```

```
    df["Date"],
```

```
    df["Closing_Price"],
```

```
    marker="o",
```

```
    label="Closing Price"
```

```
)
```

```
plt.title("Evolution of Stock Prices")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Price")
```

```
plt.legend()
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----  
# 2. Trading Volume vs Return  
# -----
```

```
plt.figure(figsize=(8,5))  
  
sns.scatterplot(  
    data=df,  
    x="Trading_Volume",  
    y="Daily_Return",  
    s=100  
)  
  
plt.title(  
    "Trading Volume vs Daily Return"  
)  
  
plt.xlabel("Trading Volume")  
plt.ylabel("Daily Return (%)")  
  
plt.tight_layout()  
plt.show()
```

```
# -----  
# 3. Return Distribution  
# -----
```

```
plt.figure(figsize=(8,5))  
  
sns.histplot(  
    df["Daily_Return"],  
    bins=8,
```

```

        kde=True
    )

plt.title("Distribution of Daily Returns")
plt.xlabel("Daily Return (%)")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# -----
# 4. Trading Volume Distribution
# -----

plt.figure(figsize=(8,5))

sns.histplot(
    df["Trading_Volume"],
    bins=8,
    kde=True
)

plt.title("Distribution of Trading Volume")
plt.xlabel("Trading Volume")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# -----
# 5. Scatter Matrix
# -----

```

```
features = [  
    "Opening_Price",  
    "Closing_Price",  
    "Trading_Volume",  
    "Daily_Return"  
]
```

```
scatter_matrix(  
    df[features],  
    figsize=(12,10),  
    diagonal="hist"  
)
```

```
plt.suptitle(  
    "Financial Market Scatter Matrix",  
    y=1.02  
)
```

```
plt.show()
```

```
# -----  
# 6. Lag Plot  
# -----
```

```
plt.figure(figsize=(7,5))
```

```
lag_plot(  
    df["Closing_Price"]  
)
```

```
plt.title(  

```

```

        "Lag Plot of Closing Prices"
    )

plt.tight_layout()
plt.show()

# -----
# 7. Autocorrelation Plot
# -----

plt.figure(figsize=(10,5))

autocorrelation_plot(
    df["Closing_Price"]
)

plt.title(
    "Autocorrelation of Closing Prices"
)

plt.tight_layout()
plt.show()

# -----
# 8. Correlation Matrix
# -----

correlation = df[
    "Opening_Price",
    "Closing_Price",
    "Trading_Volume",

```

```

        "Daily_Return"
    ]
].corr()

print("\nCORRELATION MATRIX")
print(correlation)

plt.figure(figsize=(8,5))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title(
    "Financial Market Correlation Matrix"
)

plt.tight_layout()
plt.show()

# -----
# 9. Summary
# -----

print("\nMARKET SUMMARY")

print(
    "Average Daily Return:",
    round(df["Daily_Return"].mean(), 2), "%"

```



```
)

print(
    "Average Trading Volume:",
    round(df["Trading_Volume"].mean(), 2)
)
```

```
print(
    "Highest Closing Price:",
    df["Closing_Price"].max()
)
```

```
print(
    "Lowest Closing Price:",
    df["Closing_Price"].min()
)
```

```
print(
    "Highest Daily Return:",
    df["Daily_Return"].max(), "%"
)
```

```
print(
    "Lowest Daily Return:",
    df["Daily_Return"].min(), "%"
)
```

OUTPUT

1. Price Evolution

The opening and closing prices show an overall upward trend during the observation period.

The closing price increases from **101.50** at the beginning to **118.00** at the end.

2. Trading Volume and Returns

The scatter plot compares trading volume with daily returns.

Higher trading volumes are observed during several periods of larger price movements, suggesting that trading activity may increase during stronger market movements.

3. Distribution of Returns

The daily return distribution contains both positive and negative values. Most daily returns are relatively close to zero, while a few larger positive and negative movements are observed.

4. Scatter Matrix

The scatter matrix compares:

- Opening Price
- Closing Price
- Trading Volume
- Daily Return

It provides a multivariable view of relationships within the financial dataset.

5. Lag Plot

The lag plot of closing prices shows a strong relationship between consecutive closing-price observations.

This indicates that today's closing price is related to the previous day's closing price.

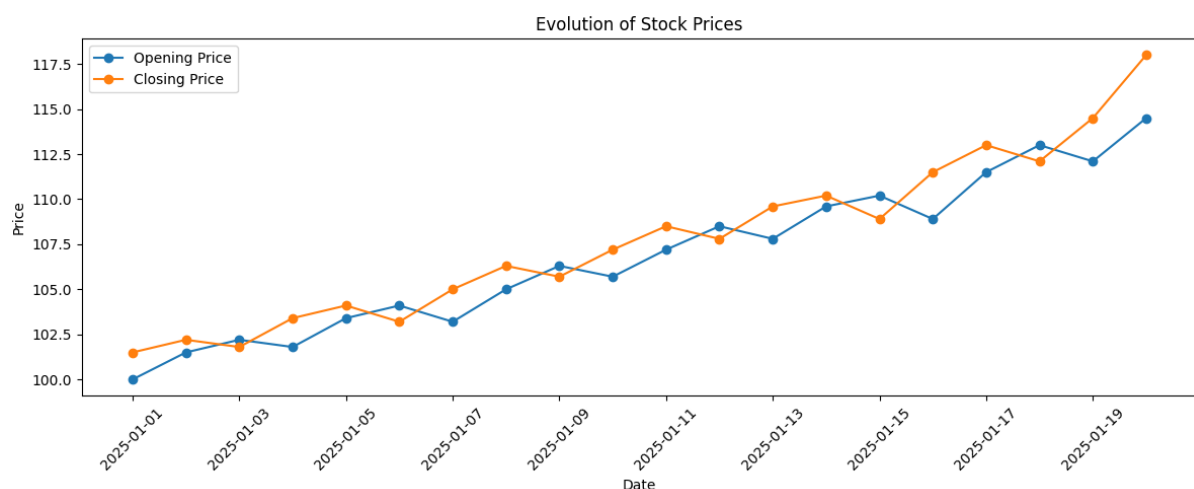
6. Autocorrelation Plot

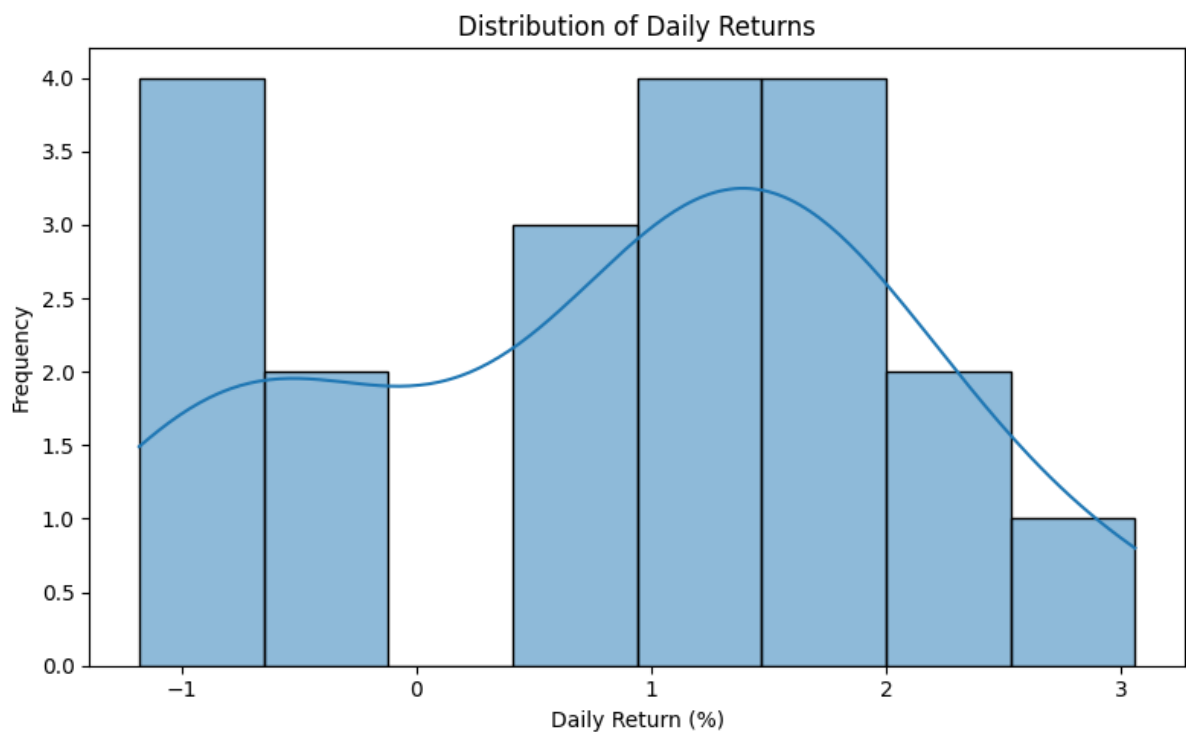
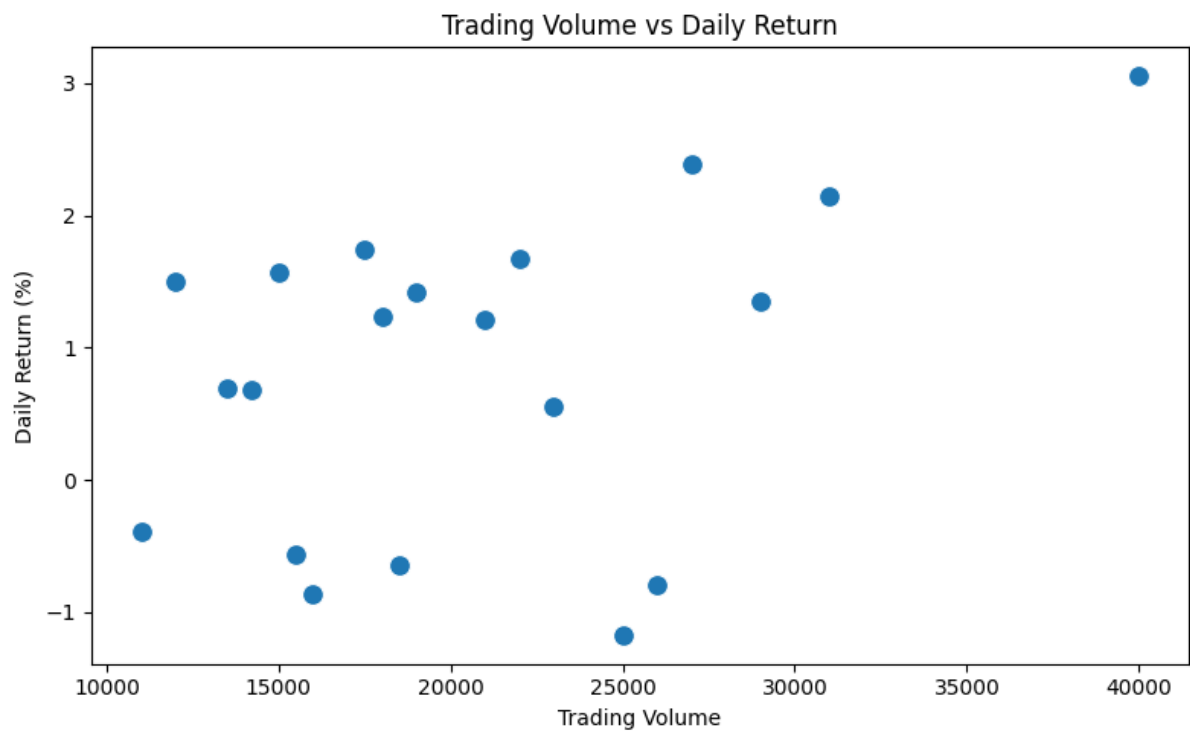
The autocorrelation plot helps identify time-dependent relationships in closing prices.

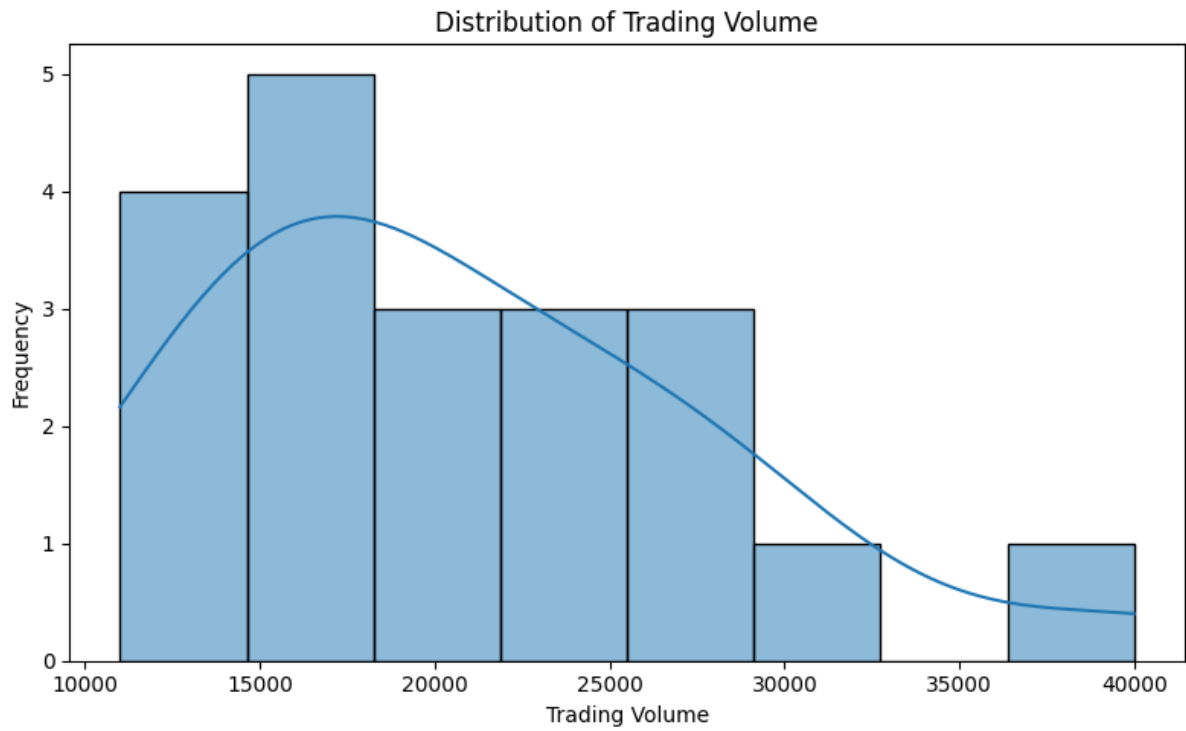
High autocorrelation at smaller lags indicates that the price series has temporal dependence.

7. Correlation Analysis

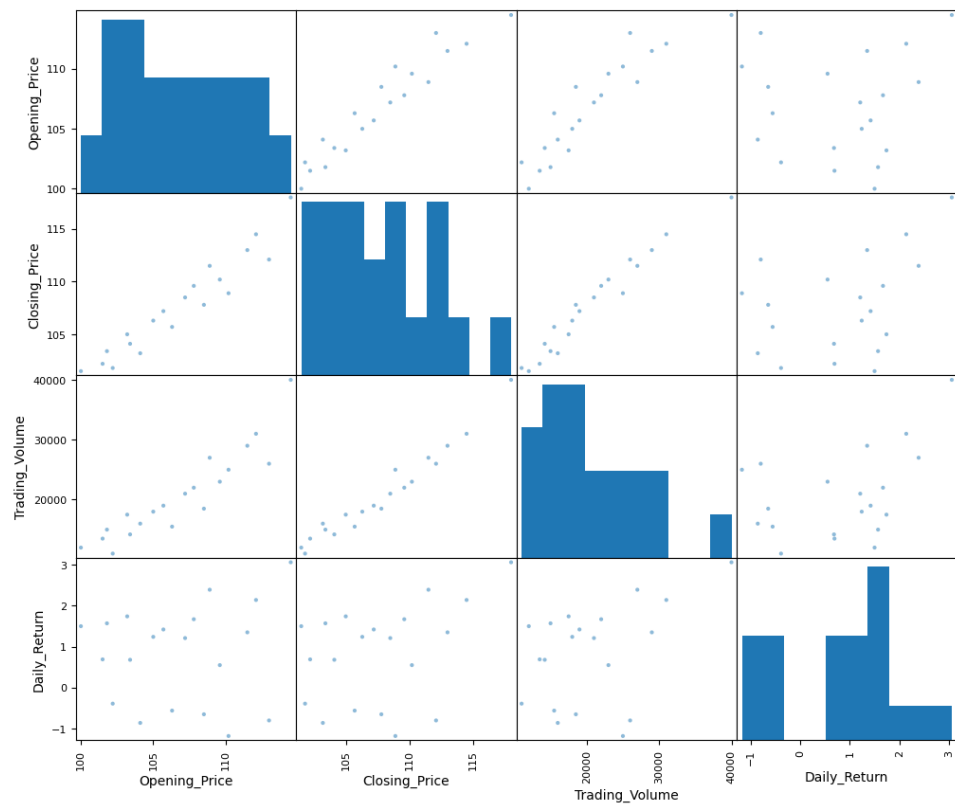
The correlation matrix helps investment analysts understand relationships between prices, trading volume, and daily returns.

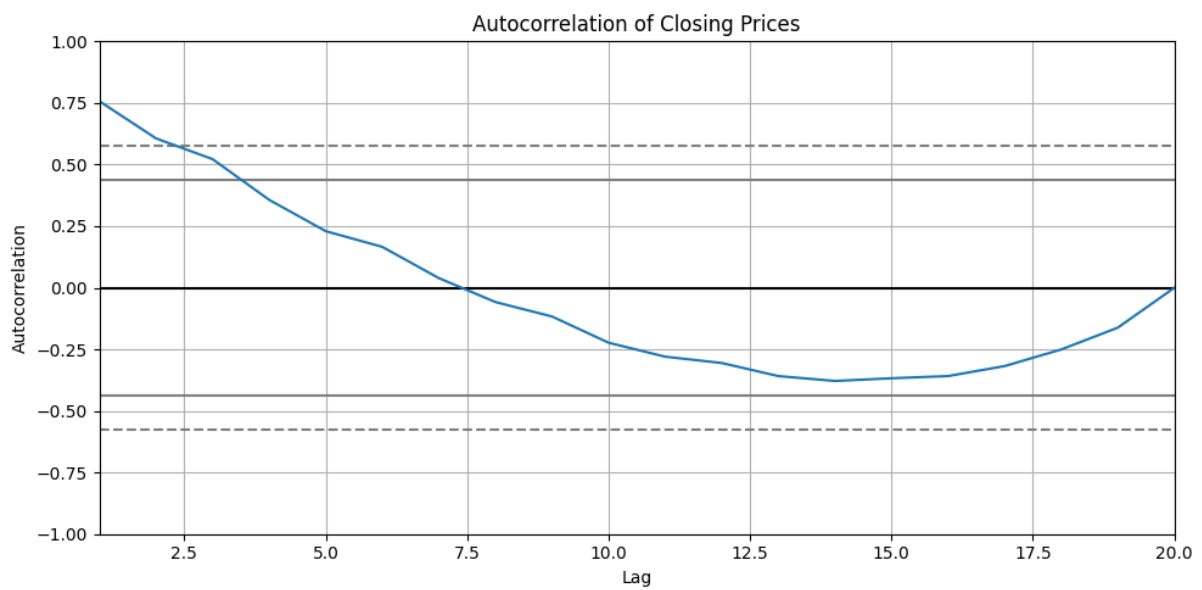
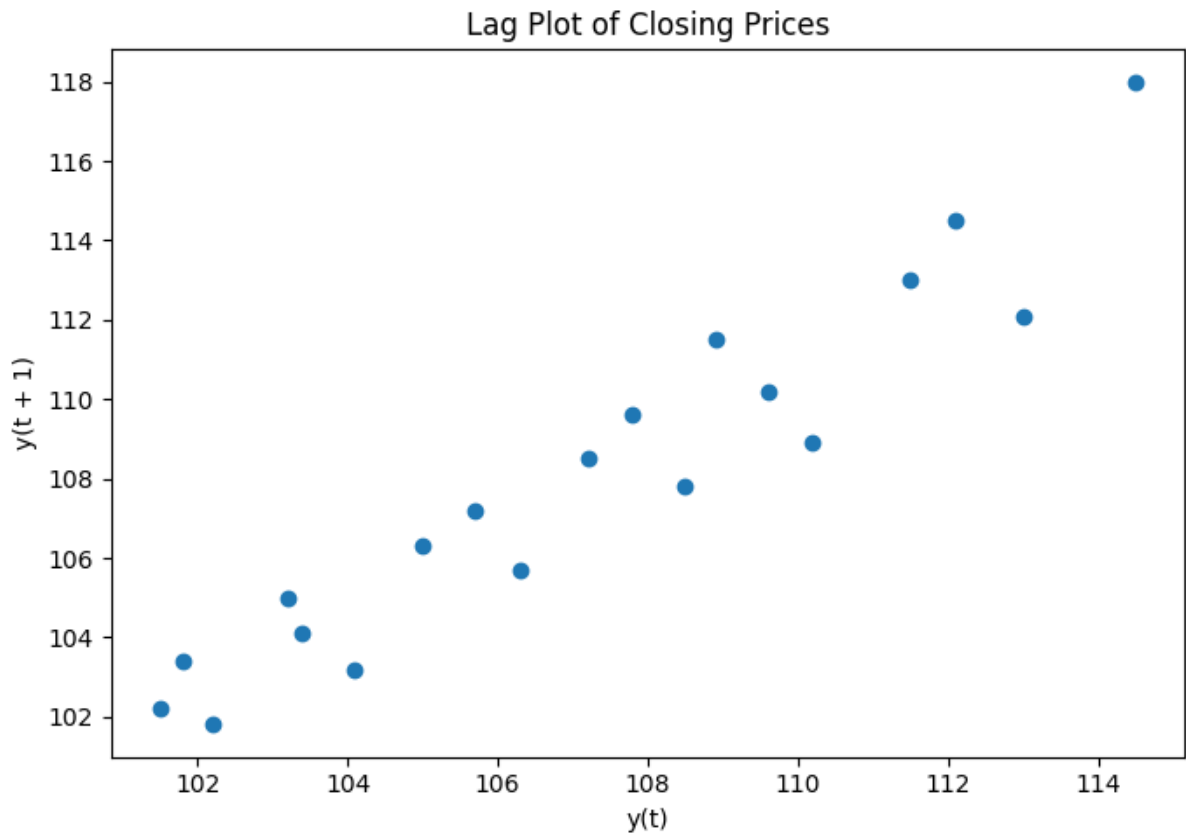






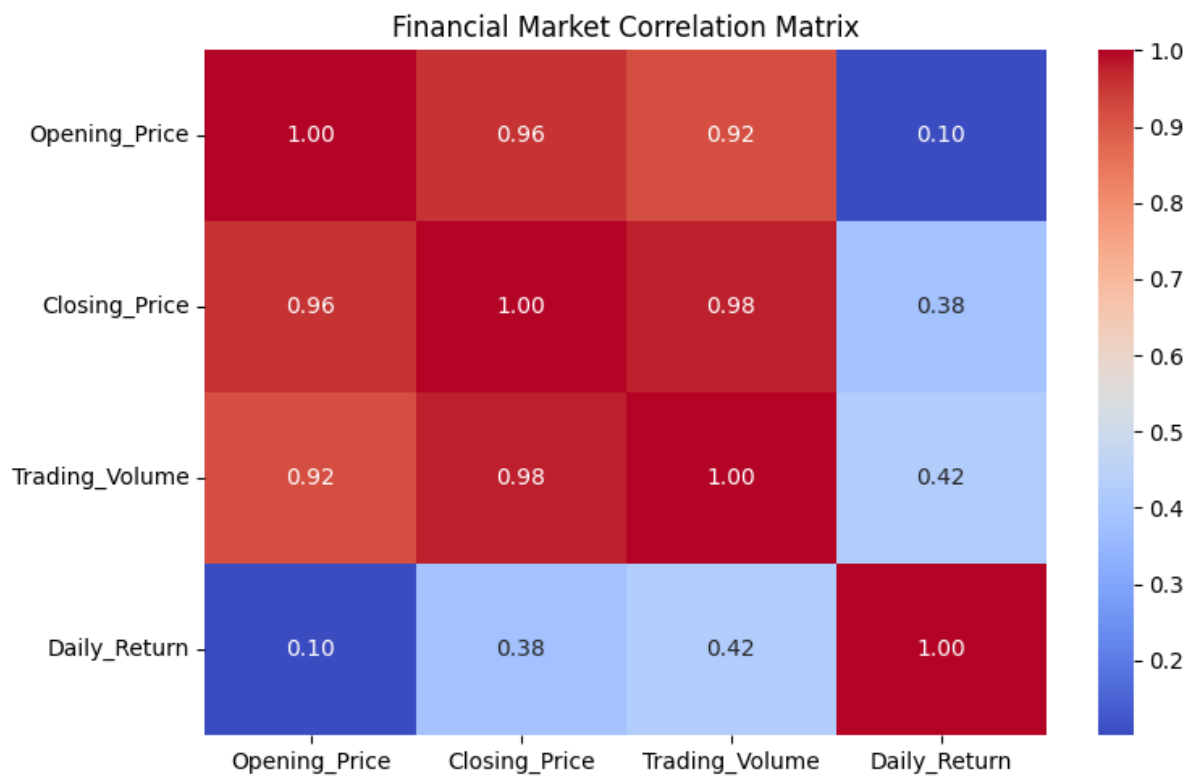
Financial Market Scatter Matrix





CORRELATION MATRIX

	Opening_Price	Closing_Price	Trading_Volume	Daily_Return
Opening_Price	1.000000	0.957591	0.918886	0.101611
Closing_Price	0.957591	1.000000	0.976230	0.383764
Trading_Volume	0.918886	0.976230	1.000000	0.423652
Daily_Return	0.101611	0.383764	0.423652	1.000000



MARKET SUMMARY

Average Daily Return: 0.84 %

Average Trading Volume: 20710.0

Highest Closing Price: 118.0

Lowest Closing Price: 101.5

Highest Daily Return: 3.06 %

Lowest Daily Return: -1.18 %

RESULT

The financial market dataset was successfully analyzed using price trends, distributions, scatter plots, scatter matrices, lag plots, and autocorrelation plots.

The closing price shows an overall upward movement during the study period. Consecutive closing prices demonstrate strong time dependence, while trading volume changes with market activity.

PROJECT 5: EMPLOYEE PRODUCTIVITY ANALYSIS

QUESTION

How do working hours, experience, salary, tasks completed, and monthly performance affect employee productivity, and what patterns and variability can be identified using distributions, scatter matrices, lag plots, autocorrelation plots, and bootstrap analysis?

AIM

To import and analyze employee productivity data using Pandas, visualize relationships between working hours, experience, and productivity, analyze distributions and frequencies, use scatter matrices, lag plots, autocorrelation plots, and bootstrap analysis to identify productivity patterns and variability, and provide data-driven recommendations to management.

DATASET

Dataset Name: Employee Productivity Dataset

Source: Representative Open HR Dataset

Number of Records: 20

Variables:

- Employee_ID
- Working_Hours
- Experience
- Salary
- Tasks_Completed
- Productivity_Score
- Monthly_Performance

Dataset Values

S.No	Employee ID	Working Hours	Experience	Salary	Tasks Completed	Productivity Score	Monthly Performance
1	E001	8	1	30000	18	65	68
2	E002	9	2	35000	21	72	74
3	E003	8	3	40000	23	75	77
4	E004	10	4	45000	26	82	84
5	E005	9	5	50000	28	85	87
6	E006	8	2	34000	20	70	72
7	E007	7	1	28000	16	60	62
8	E008	10	6	58000	30	88	90

9	E009	9	7	62000	32	90	92
10	E010	8	4	47000	25	78	80
11	E011	11	8	68000	35	92	94
12	E012	9	3	41000	24	76	78
13	E013	7	2	30000	17	58	60
14	E014	10	5	55000	29	86	88
15	E015	12	10	80000	40	95	97
16	E016	8	3	39000	22	73	75
17	E017	9	6	57000	31	87	89
18	E018	7	1	27000	15	55	57
19	E019	10	8	65000	34	91	93
20	E020	14	12	15000 0	50	98	99

CODE

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import numpy as np
```

```
from pandas.plotting import (
```

```
    scatter_matrix,
```

```
    lag_plot,
```

```
    autocorrelation_plot
```

```
)
```

```
# Read dataset
```

```
df = pd.read_csv(
```

```
    "employee_productivity_monthly.csv"
```

```
)
```

```
print("EMPLOYEE PRODUCTIVITY DATASET")
```

```
print(df)
```



```
# -----  
# 1. Descriptive Statistics  
# -----  
  
print("\nDESCRIPTIVE STATISTICS")  
print(df.describe())
```

```
# -----  
# 2. Working Hours Distribution  
# -----
```

```
plt.figure(figsize=(8,5))  
  
sns.histplot(  
    df["Working_Hours"],  
    bins=8,  
    kde=True  
)  
  
plt.title(  
    "Distribution of Working Hours"  
)
```

```
plt.xlabel("Working Hours")  
plt.ylabel("Frequency")
```

```
plt.tight_layout()  
plt.show()
```

```
# -----  
# 3. Productivity Distribution
```

```
# -----

plt.figure(figsize=(8,5))

sns.histplot(
    df["Productivity_Score"],
    bins=8,
    kde=True
)

plt.title(
    "Distribution of Productivity Scores"
)

plt.xlabel("Productivity Score")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

```
# -----
# 4. Performance Frequency
# -----
```

```
plt.figure(figsize=(8,5))

sns.histplot(
    df["Monthly_Performance"],
    bins=8,
    kde=True
)
```

```
plt.title(  
    "Monthly Performance Distribution"  
)
```

```
plt.xlabel("Monthly Performance")  
plt.ylabel("Frequency")
```

```
plt.tight_layout()  
plt.show()
```

```
# -----  
# 5. Working Hours vs Productivity  
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(  
    data=df,  
    x="Working_Hours",  
    y="Productivity_Score",  
    s=100  
)
```

```
plt.title(  
    "Working Hours vs Productivity"  
)
```

```
plt.xlabel("Working Hours")  
plt.ylabel("Productivity Score")
```

```
plt.tight_layout()  
plt.show()
```

```
# -----  
# 6. Experience vs Productivity  
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(  
    data=df,  
    x="Experience",  
    y="Productivity_Score",  
    s=100  
)
```

```
plt.title(  
    "Experience vs Productivity"  
)
```

```
plt.xlabel("Experience (Years)")  
plt.ylabel("Productivity Score")
```

```
plt.tight_layout()  
plt.show()
```

```
# -----  
# 7. Tasks vs Productivity  
# -----
```

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(  
    data=df,
```

```

        x="Tasks_Completed",
        y="Productivity_Score",
        s=100
    )

plt.title(
    "Tasks Completed vs Productivity"
)

plt.xlabel("Tasks Completed")
plt.ylabel("Productivity Score")

plt.tight_layout()
plt.show()

# -----
# 8. Scatter Matrix
# -----

features = [
    "Working_Hours",
    "Experience",
    "Salary",
    "Tasks_Completed",
    "Productivity_Score",
    "Monthly_Performance"
]

scatter_matrix(
    df[features],
    figsize=(14,12),
    diagonal="hist"

```

```
)
```

```
plt.suptitle(  
    "Employee Productivity Scatter Matrix",  
    y=1.02  
)
```

```
plt.show()
```

```
# -----  
# 9. Lag Plot  
# -----
```

```
plt.figure(figsize=(7,5))
```

```
lag_plot(  
    df["Productivity_Score"]  
)
```

```
plt.title(  
    "Lag Plot of Productivity Scores"  
)
```

```
plt.tight_layout()  
plt.show()
```

```
# -----  
# 10. Autocorrelation Plot  
# -----
```

```
plt.figure(figsize=(10,5))
```

```
autocorrelation_plot(  
    df["Productivity_Score"]  
)  
  
plt.title(  
    "Autocorrelation of Productivity Scores"  
)
```

```
plt.tight_layout()  
plt.show()
```

```
# -----  
# 11. Bootstrap Analysis  
# -----
```

```
np.random.seed(42)
```

```
bootstrap_means = []
```

```
for i in range(1000):
```

```
    sample = np.random.choice(  
        df["Productivity_Score"],  
        size=len(df),  
        replace=True  
    )
```

```
    bootstrap_means.append(  
        np.mean(sample)  
    )
```

```
bootstrap_means = np.array(  
    bootstrap_means
```

```

        bootstrap_means
    )

    lower = np.percentile(
        bootstrap_means,
        2.5
    )

    upper = np.percentile(
        bootstrap_means,
        97.5
    )

    print("\nBOOTSTRAP ANALYSIS")

    print(
        "Original Mean Productivity:",
        round(
            df["Productivity_Score"].mean(),
            2
        )
    )

    print(
        "95% Bootstrap Confidence Interval:",
        round(lower, 2),
        "to",
        round(upper, 2)
    )

    # Bootstrap Plot
    plt.figure(figsize=(9,5))

```



```
plt.hist(  
    bootstrap_means,  
    bins=30  
)
```

```
plt.axvline(  
    lower,  
    linestyle="--",  
    label="2.5% Limit"  
)
```

```
plt.axvline(  
    upper,  
    linestyle="--",  
    label="97.5% Limit"  
)
```

```
plt.axvline(  
    bootstrap_means.mean(),  
    linestyle="-",  
    label="Bootstrap Mean"  
)
```

```
plt.title(  
    "Bootstrap Distribution of Mean Productivity"  
)
```

```
plt.xlabel("Mean Productivity")  
plt.ylabel("Frequency")
```

```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 12. Correlation Matrix
```

```
# -----
```

```
correlation = df[
```

```
    features
```

```
].corr()
```

```
print("\nCORRELATION MATRIX")
```

```
print(correlation)
```

```
plt.figure(figsize=(10,7))
```

```
sns.heatmap(
```

```
    correlation,
```

```
    annot=True,
```

```
    cmap="coolwarm",
```

```
    fmt=".2f"
```

```
)
```

```
plt.title(
```

```
    "Employee Productivity Correlation Matrix"
```

```
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 13. Recommendations
```

```
print("\nDATA-DRIVEN RECOMMENDATIONS")
```

```
print("1. Provide training and mentoring for less experienced employees.")
```

```
print("2. Monitor employees working very long hours.")
```

```
print("3. Recognize and reward consistently high-performing employees.")
```

```
print("4. Balance task allocation to avoid employee overload.")
```

```
print("5. Use productivity trends for workforce planning.")
```

OUTPUT

1. Working Hours Distribution

Most employees work between **7 and 10 hours**. Employee E020 works 14 hours, which is considerably higher than most employees.

2. Productivity Distribution

Most productivity scores are between **60 and 95**. Higher productivity scores are generally associated with employees having greater experience and completing more tasks.

3. Experience vs Productivity

The scatter plot shows a positive relationship between experience and productivity.

Employees with more years of experience generally achieve higher productivity scores.

4. Working Hours vs Productivity

Productivity generally increases with working hours in the dataset. However, extremely long working hours should not automatically be interpreted as desirable because they may indicate employee overload.

5. Tasks Completed vs Productivity

Employees completing more tasks generally have higher productivity scores.

6. Scatter Matrix

The scatter matrix provides a combined view of:

- Working Hours
- Experience
- Salary
- Tasks Completed
- Productivity Score
- Monthly Performance

It helps identify relationships between multiple employee variables.

7. Lag Plot

The lag plot compares each employee's productivity score with the previous observation.

Because employee records are not a true chronological time series, the lag plot should be interpreted as **record-order dependence**, not as a genuine time-series relationship.

8. Autocorrelation Plot

The autocorrelation plot examines dependence between productivity scores at different record lags.

Since the dataset is employee-level rather than a continuous time series, this analysis is mainly exploratory.

9. Bootstrap Analysis

A bootstrap sample of **1,000 resamples** is generated to estimate the variability of the mean productivity score.

The 95% bootstrap confidence interval provides an estimated range for the population mean productivity.

10. Unusual Employee Record

Employee **E020** is an unusual observation:

- Working Hours: 14
- Experience: 12 years
- Salary: ₹150,000
- Tasks Completed: 50
- Productivity Score: 98
- Monthly Performance: 99

This employee may be a highly experienced senior employee, but the unusually high working hours should also be investigated to ensure that productivity is not dependent on excessive workload.

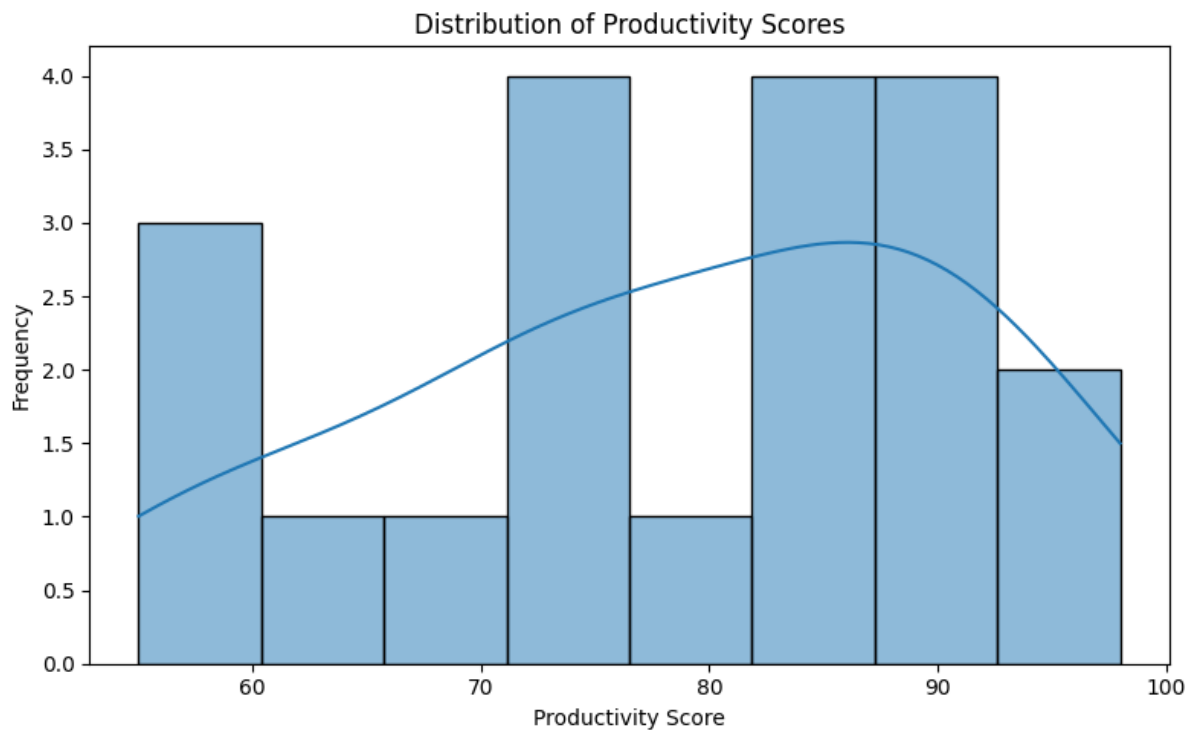
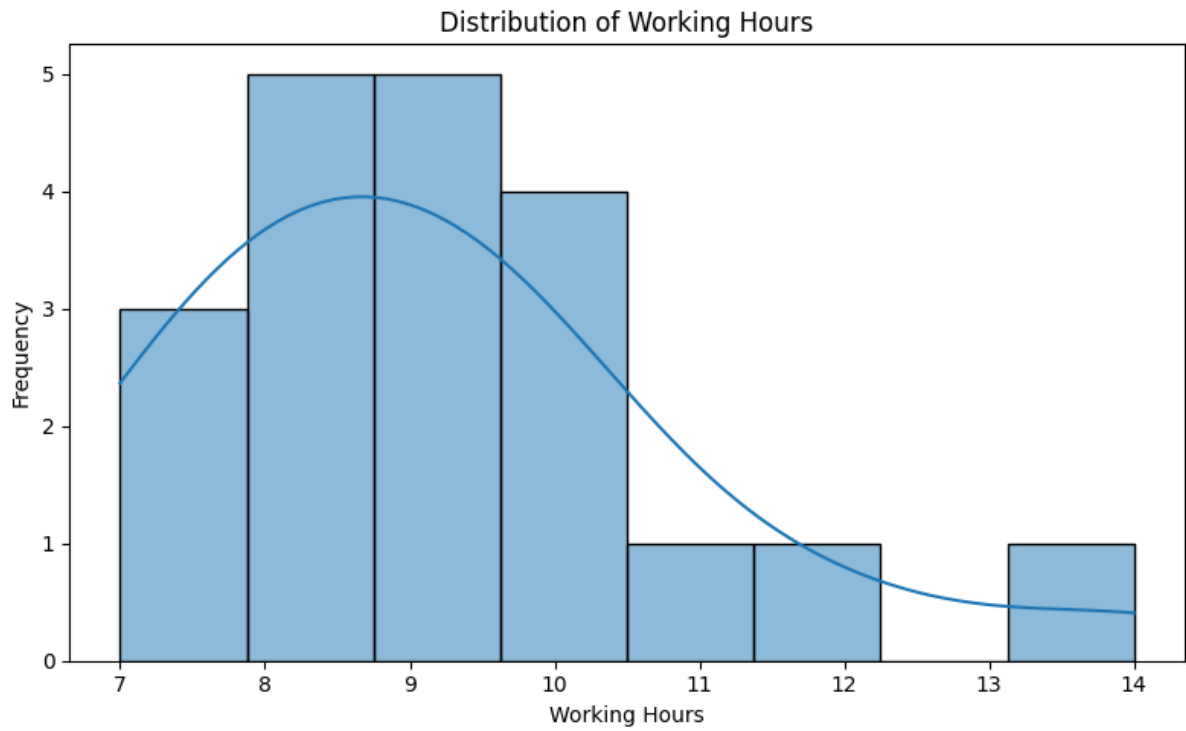
	Productivity_Score	Monthly_Performance
0	65	68
1	72	74
2	75	77
3	82	84
4	85	87
5	70	72
6	60	62
7	88	90
8	90	92
9	78	80

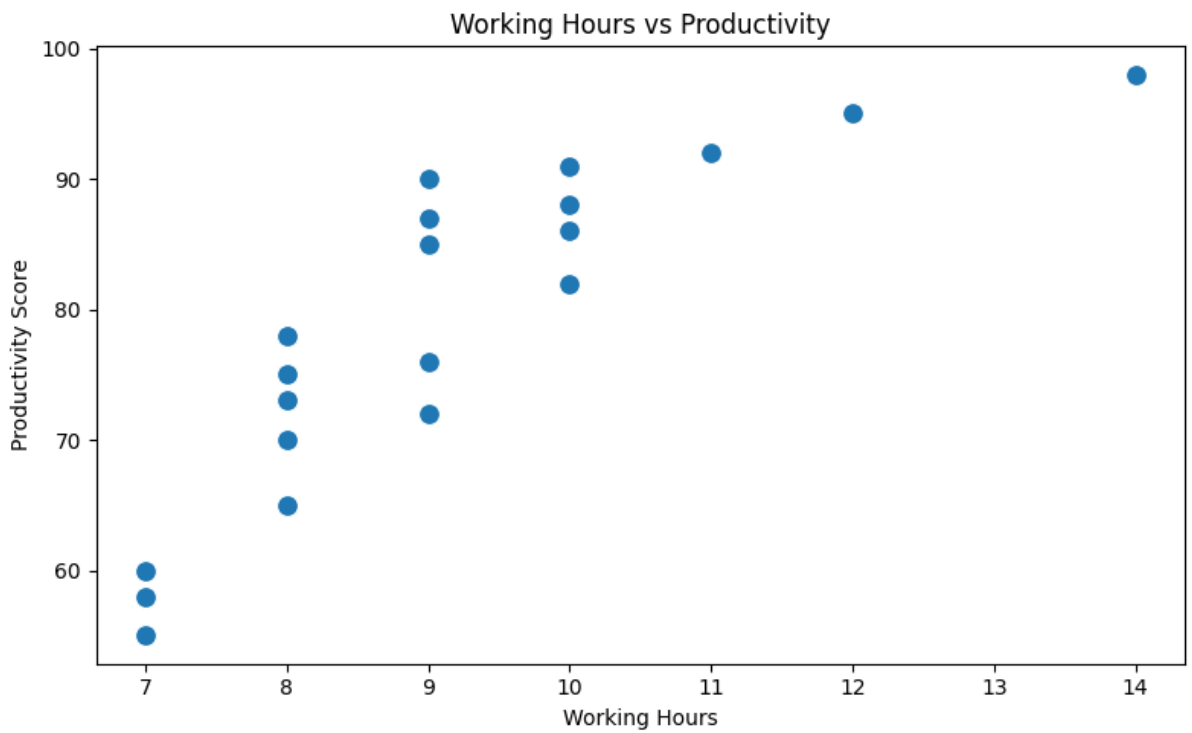
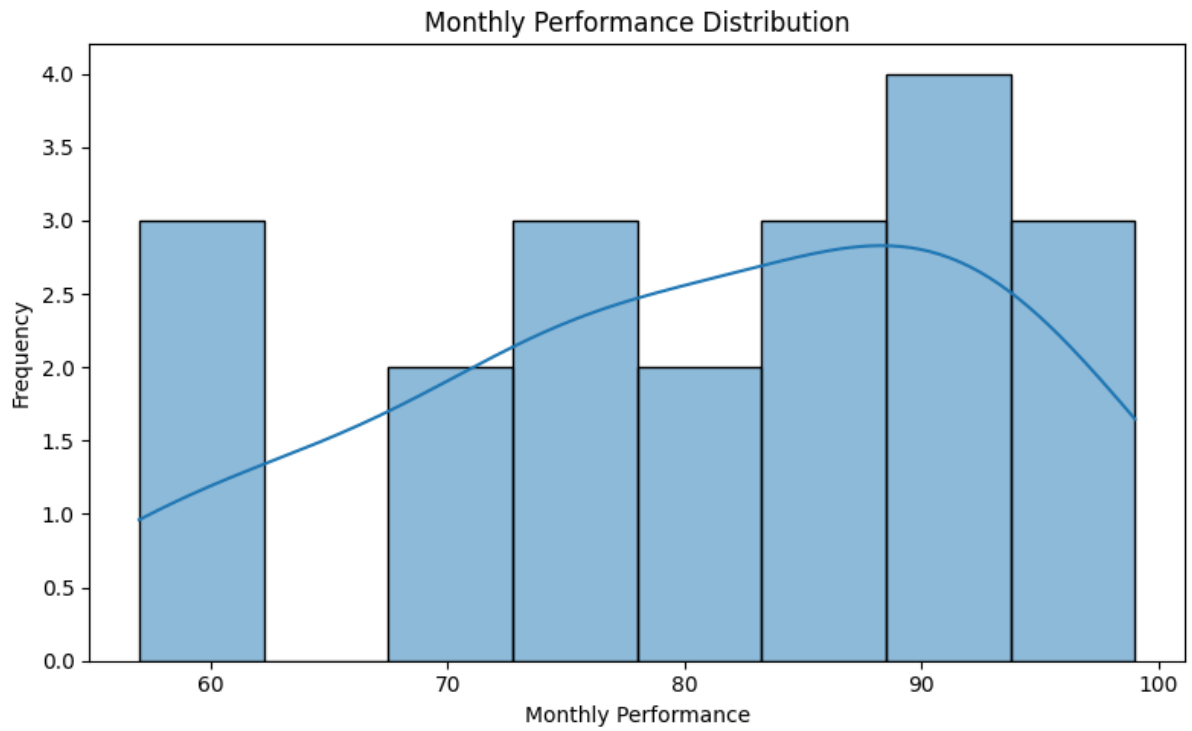
10	92	94
11	76	78
12	58	60
13	86	88
14	95	97
15	73	75
16	87	89
17	55	57
18	91	93
19	98	99

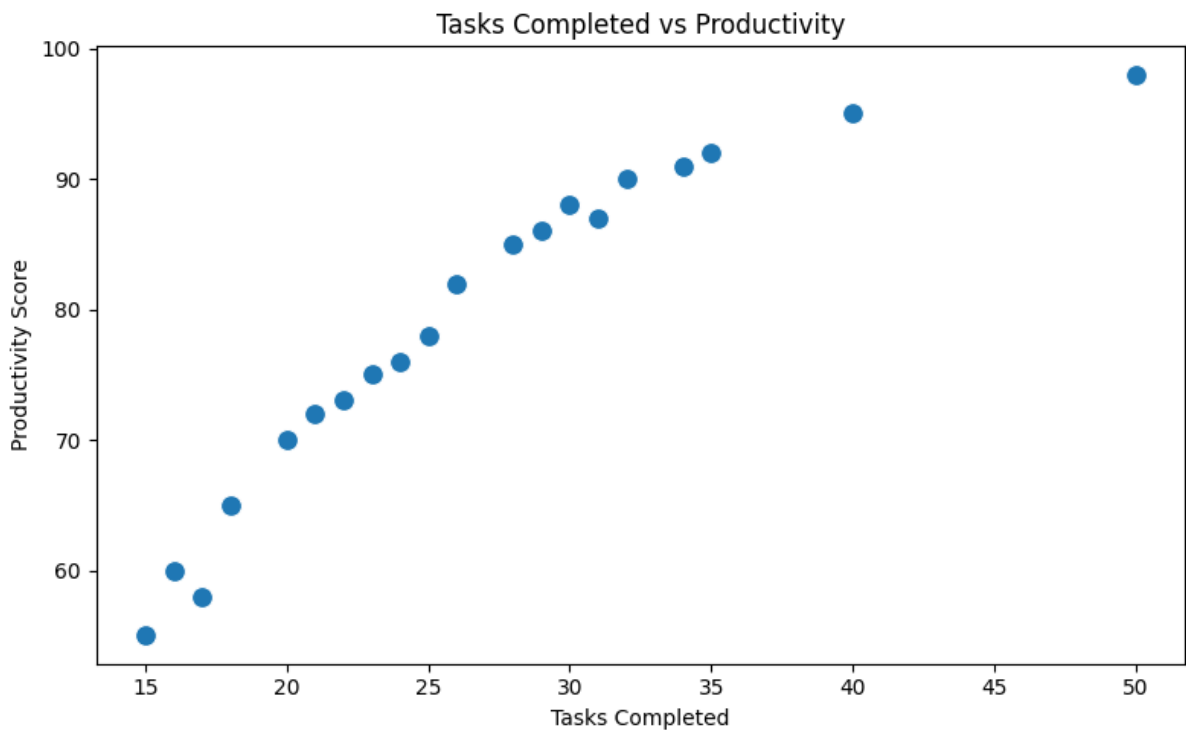
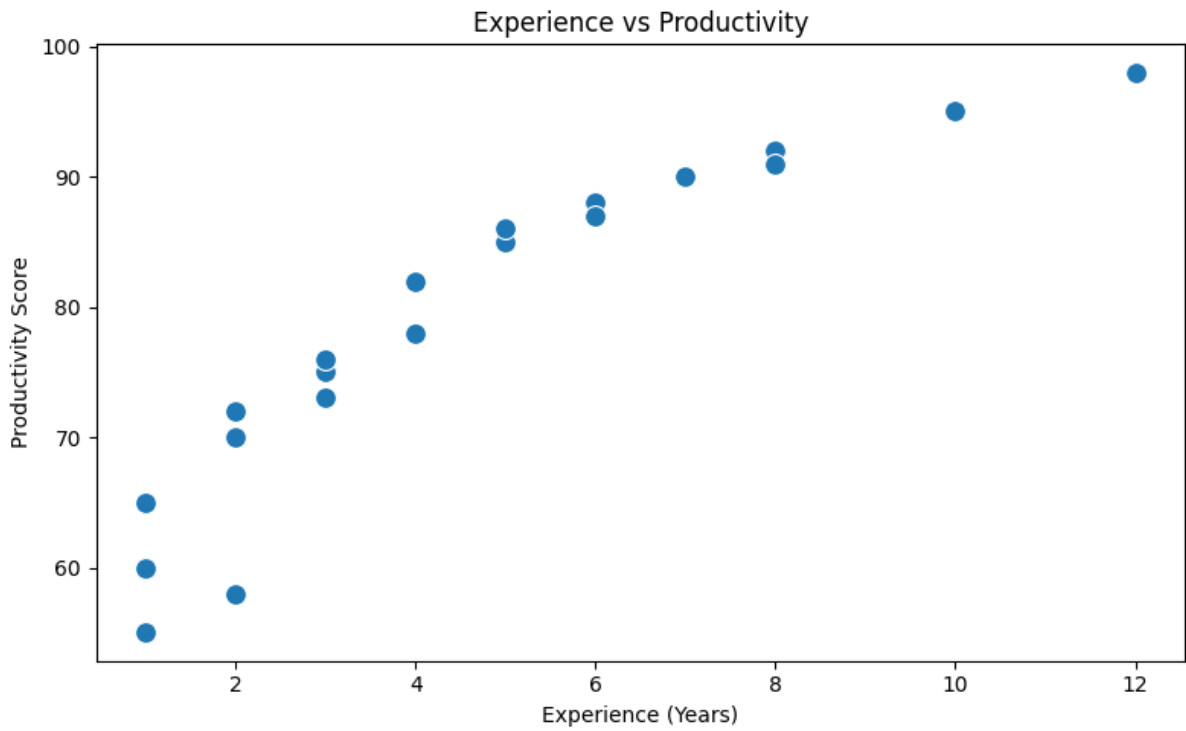
DESCRIPTIVE STATISTICS

	Working_Hours	Experience	Salary	Tasks_Completed \
count	20.000000	20.000000	20.000000	20.000000
mean	9.150000	4.650000	52050.00000	26.800000
std	1.755443	3.116594	27412.01235	8.739505
min	7.000000	1.000000	27000.00000	15.000000
25%	8.000000	2.000000	34750.00000	20.750000
50%	9.000000	4.000000	46000.00000	25.500000
75%	10.000000	6.250000	59000.00000	31.250000
max	14.000000	12.000000	150000.00000	50.000000

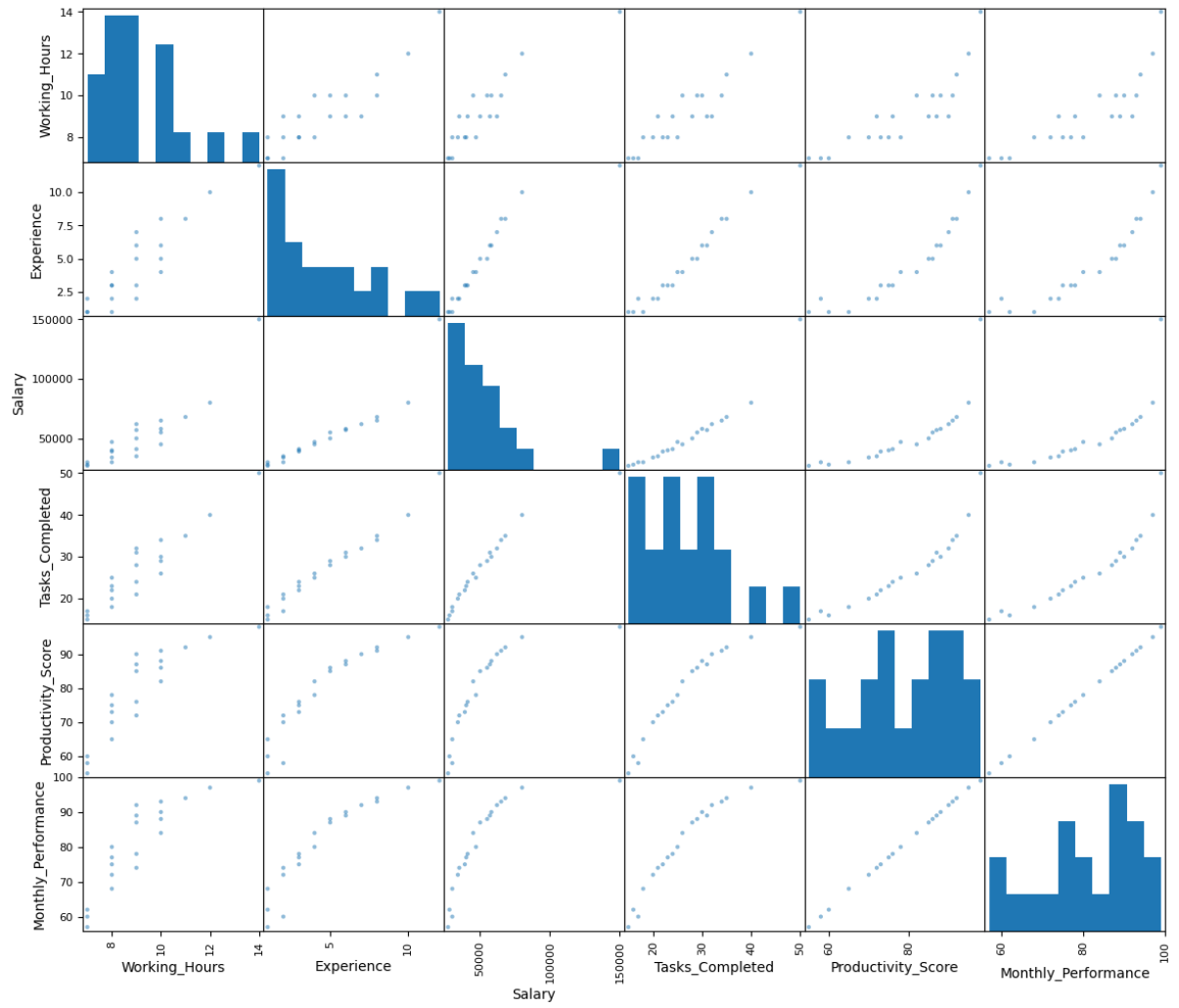
	Productivity_Score	Monthly_Performance
count	20.000000	20.000000
mean	78.800000	80.800000
std	12.680694	12.547174
min	55.000000	57.000000
25%	71.500000	73.500000
50%	80.000000	82.000000
75%	88.500000	90.500000
max	98.000000	99.000000

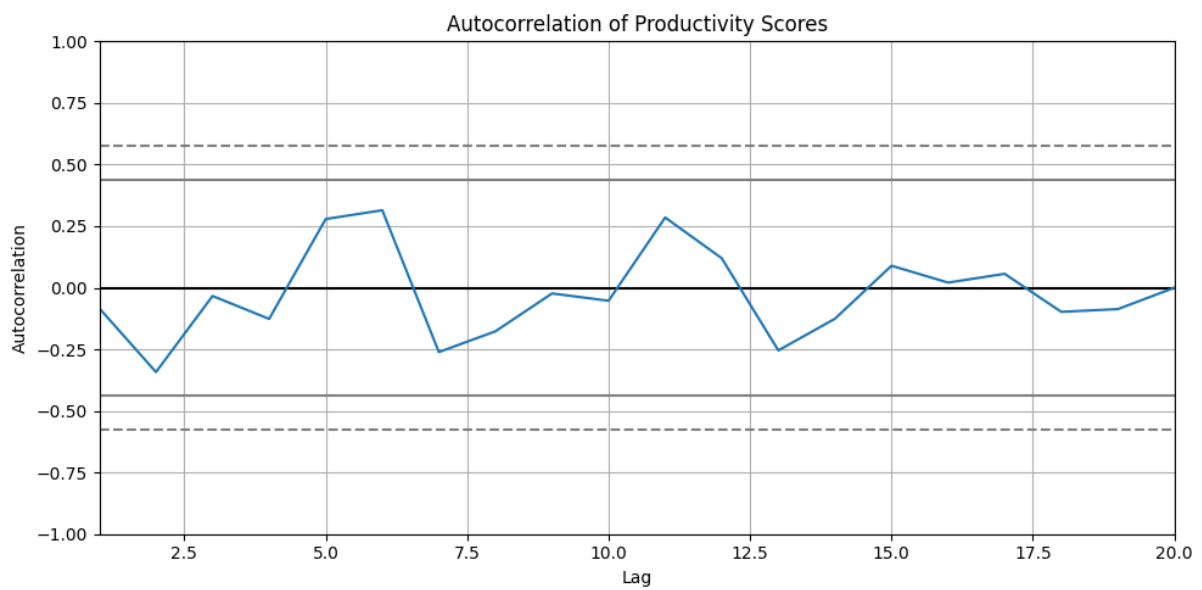
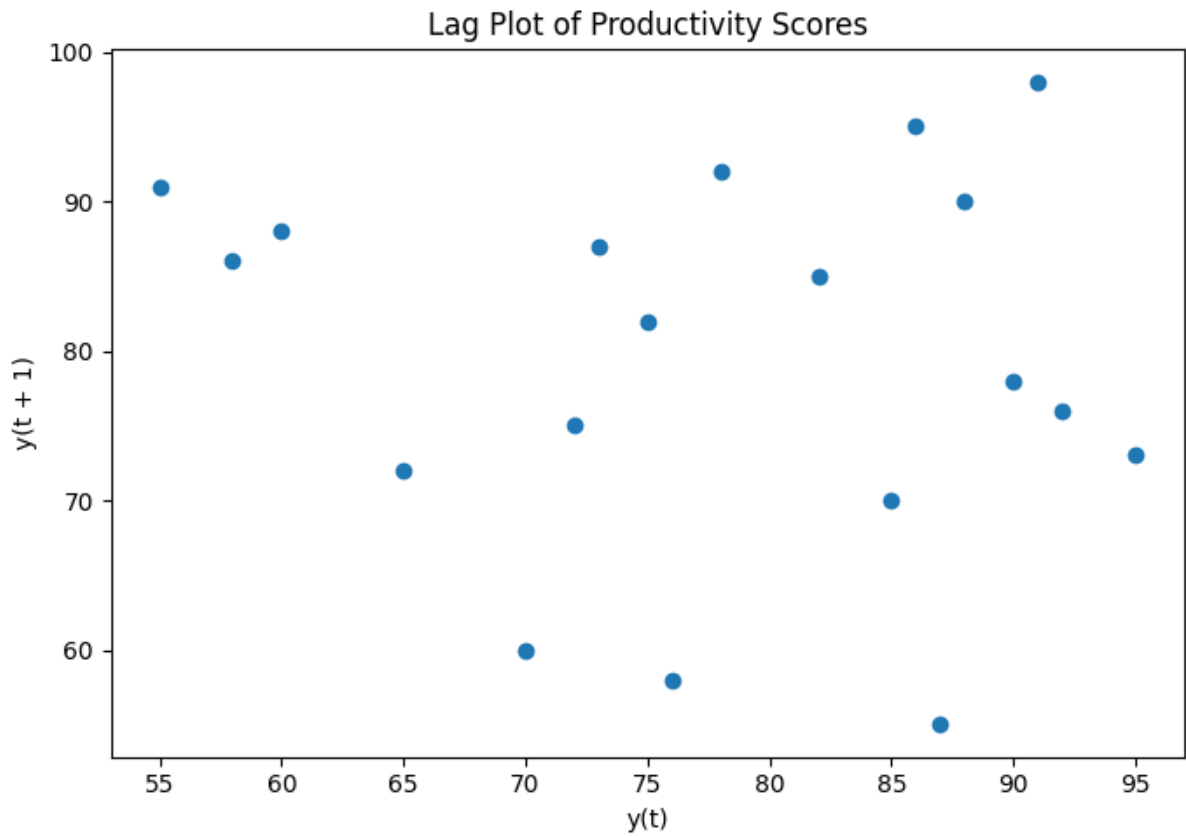






Employee Productivity Scatter Matrix

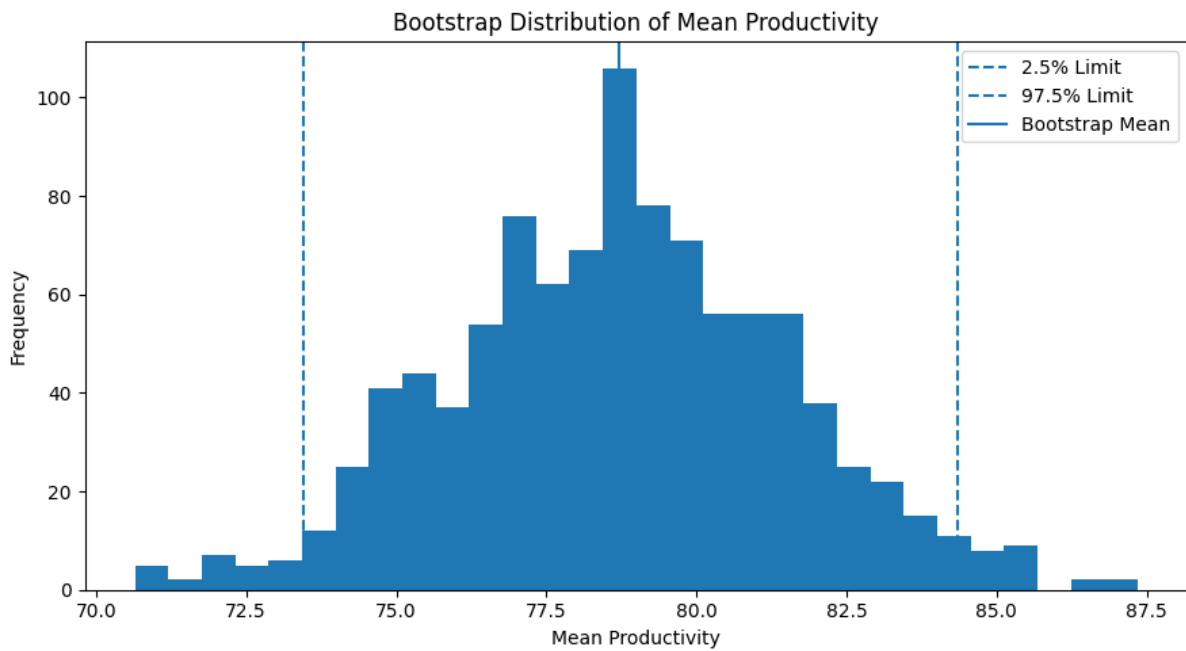




BOOTSTRAP ANALYSIS

Original Mean Productivity: 78.8

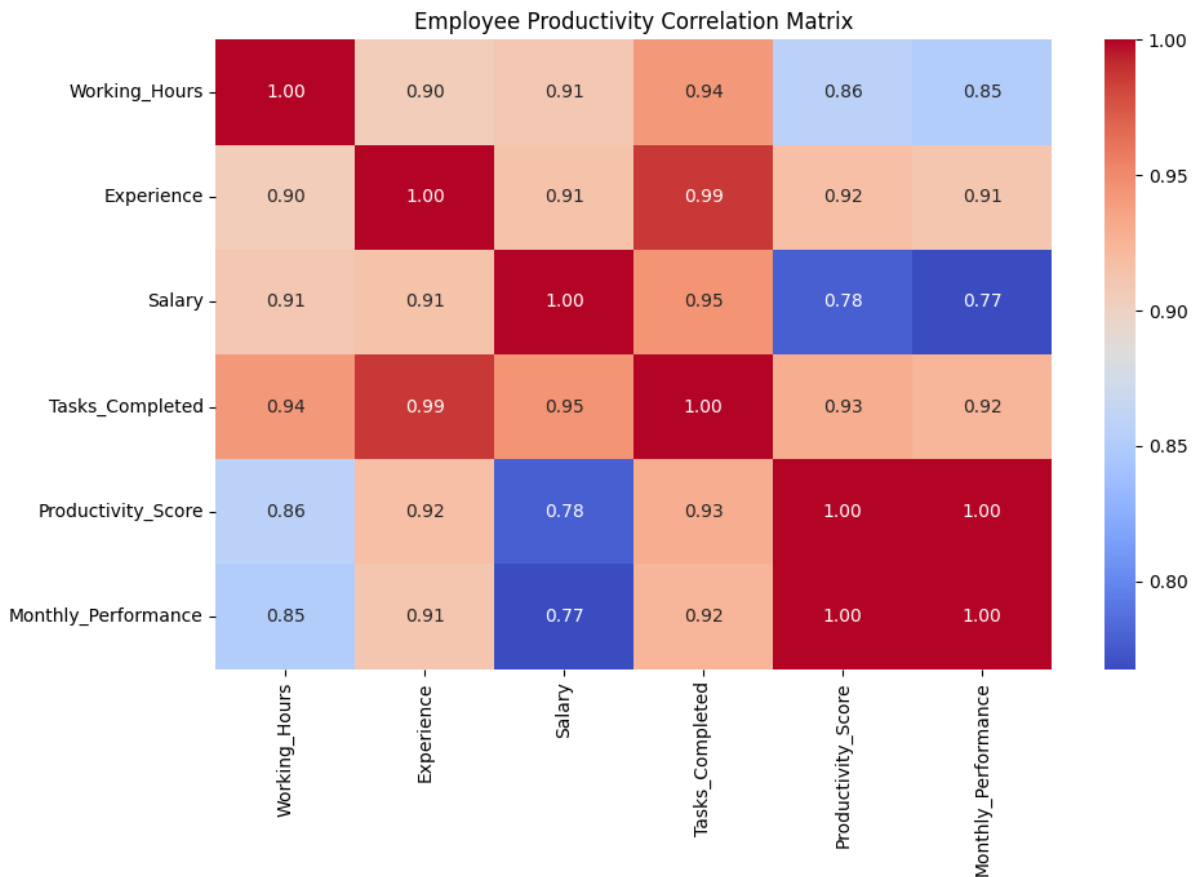
95% Bootstrap Confidence Interval: 73.45 to 84.35



CORRELATION MATRIX

	Working_Hours	Experience	Salary	Tasks_Completed \
Working_Hours	1.000000	0.904770	0.910931	0.942049
Experience	0.904770	1.000000	0.913837	0.986643
Salary	0.910931	0.913837	1.000000	0.945169
Tasks_Completed	0.942049	0.986643	0.945169	1.000000
Productivity_Score	0.857323	0.917044	0.777838	0.929980
Monthly_Performance	0.852109	0.911997	0.767752	0.924517

	Productivity_Score	Monthly_Performance
Working_Hours	0.857323	0.852109
Experience	0.917044	0.911997
Salary	0.777838	0.767752
Tasks_Completed	0.929980	0.924517
Productivity_Score	1.000000	0.999725
Monthly_Performance	0.999725	1.000000



DATA-DRIVEN RECOMMENDATIONS

1. Provide training and mentoring for less experienced employees.
2. Monitor employees working very long hours.
3. Recognize and reward consistently high-performing employees.
4. Balance task allocation to avoid employee overload.
5. Use productivity trends for workforce planning.

RESULT

The employee productivity dataset was successfully analyzed using Pandas, distributions, frequency plots, scatter plots, scatter matrices, lag plots, autocorrelation analysis, and bootstrap sampling.

The results show positive relationships between **experience, tasks completed, and productivity**. Higher-performing employees generally have greater experience and complete more tasks.

Bootstrap analysis provides an estimate of the uncertainty around the average productivity score. E020 is identified as an unusual employee record because of the combination of high salary, long working hours, high task completion, and very high productivity.